YOLO-OBB Model Development for Slide Analysis

Project Documentation

December 4, 2025

1 Component Detection Model

I developed a custom YOLO-OBB (Oriented Bounding Box) model to help detect components on lecture slides. Since diagrams often have rotated shapes/images so standard bounding boxes aren't accurate enough. OBB allows the model to be more accurate on these types of objects.

1.1 Target Class

Right now, the model is focused on a single class to get every visual component:

- `diagram_node`**:** This includes all the visual parts of a slide, like the nodes of a diagram, shapes, images, etc...

1.2 Dataset Construction

Finding good annotated datasets for presentation slides is difficult, so I built a hybrid dataset to cover different visual styles.

1.2.1 Data Sources

- **AI2D (Scientific Diagrams):** I used the AI2D dataset to get high-quality bboxes for nodes in scientific diagrams.
- **Synthetic Generation:** To make the model work better on technical slides with flowcharts, I wrote a script to generate synthetic data. It places random elements like cloud icons, flowchart boxes, and stylized shapes onto slide-like backgrounds. This helps the model learn to recognize objects in diagrams and flowcharts more related to the computer science field.

Split	Count	Percentage
Train	6320	80%
Validation	810	10%
Test	814	10%

Table 1: Dataset Splits for Logo/Image Model

1.3 Training and Refinement

1. **Initial Training:** I started by training on the mix of AI2D and my synthetic data.
   mAP50: 0.995
   mAP50-95: 0.98

   I then ran this model on actual lecture slides from my classes. The performance was not good.

2. **Fine-Tuning:** I manually corrected the detections on 100 real slides using the model trained above. Retraining on this small "gold standard" set significantly improved the results on actual slides.
mAP50: 0.95
mAP50-95: 0.87

2 Logo vs. Image Classification

Another issue I had to solve was distinguishing between a company logo and a regular content image (like a photo or screenshot). I trained a separate model just for this.

2.1 Dataset Overview

I generated this dataset synthetically.

- **Logo (0):** Brand marks and logos.
- **Images (1):** Any other kind of image.

I used logos from the *Logo-2k+* dataset and images from the *SlideVQA* dataset. The generation script places 2 to 6 objects on a gradient background and adds random text titles. This prevents the model from relying on simple shortcuts (like assuming a logo always appears on a white background).

2.2 Results

The dataset is fairly balanced (48% logos vs 52% images). The model learned the difference very quickly.

Split	Count	Percentage
Train	1,564	70%
Validation	333	15%
Test	329	15%

Table 2: Dataset Splits for Logo/Image Model

On the test set, the performance is very good:

- **mAP50:** 0.998
- **mAP50-95:** 0.995

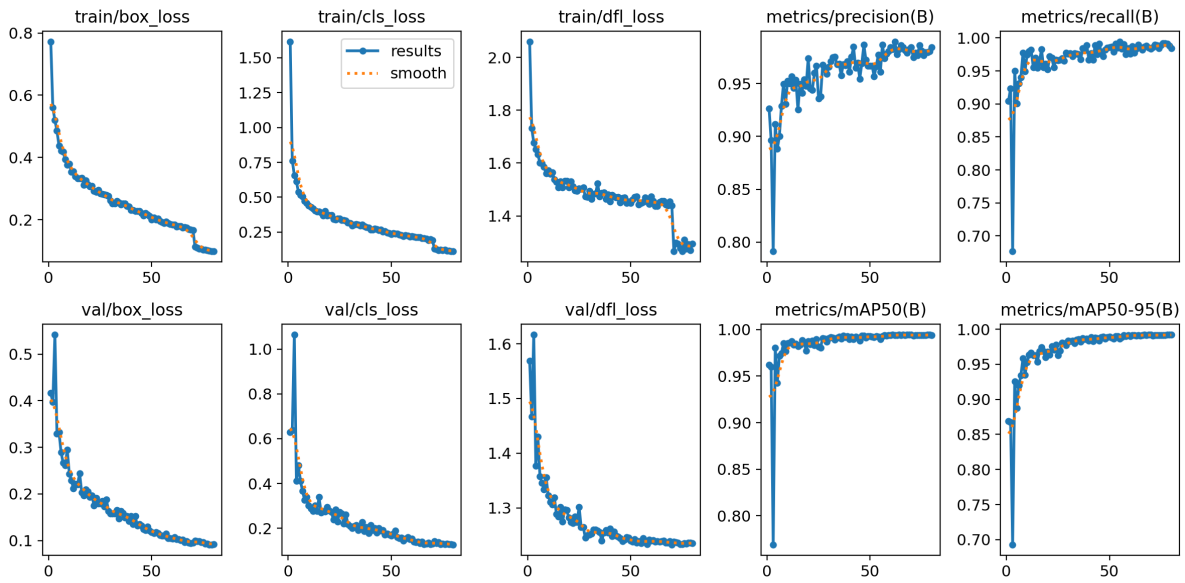

Figure 1: Training metrics for the Logo vs. Image model. As shown in the graphs, the loss drops quickly and the precision/recall scores hit nearly 1.0 very early in the training.